

RiffRush

Szczegolowy plan realizacji i wdrozenia

Dokument opisuje pelna droge od uporządkowania zakresu produktu, przez budowe lokalnego silnika audio i platformy webowej, az do bezpiecznego uruchomienia aplikacji dla pierwszych uzytkownikow oraz przygotowania procesu dalszego skalowania.

CEL Doprowadzenie produktu do stabilnego launchu

ZAKRES Web, backend, local engine, content, QA, ops

ZASADA GLOWNA Engine jest zrodlem prawdy dla audio, czasu i scoringu

Jak czytac ten plan

Kazdy etap zawiera: cel, konkretne prace do wykonania, artefakty koncowe oraz kryteria zamknienia. Etapy sa ukladane w kolejnosci realizacji. Nie nalezy rozpoczynac kolejnych obszarow dopoki poprzedni nie daje dzialajacego i sprawdzalnego rezultatu.

Definicja konca projektu

- Uzytkownik moze zalozyc konto, przejsc onboarding i pobrac local engine.
- Silnik lokalny uruchamia sie bezproblemowo, wykrywa audio i laczy sie z aplikacja webowa.
- Trening dziala end to end: start sesji, odbior zdarzen, scoring, zapis wyniku, prezentacja progresu.
- Produkt posiada monitoring, obsluge bledow, dokumentacje operacyjna i wsparcie po starcie.
- Publiczny launch mozna przeprowadzic bez recznych interwencji na kazdej sesji uzytkownika.

Faza 1 Ustalenie scope i architektury

Faza 2 Budowa dzialajacego rdzenia audio

Faza 3 Integracja produktu i testy beta

Faza 4 Gotowosc operacyjna i publiczny launch

ETAP 01

Zamknienie zakresu MVP

Celem jest decyzja, co dokladnie wchodzi do pierwszej wersji, a co pozostaje poza MVP.

PRACE DO WYKONANIA

- Zdefiniować 1 główny typ treningu i 1 główny flow sesji.
- Ograniczyć engine do monofonii i podstawowego scoringu.
- Spisać minimalny onboarding, dashboard i widok wyniku.
- Zamknąć listę platform startowych, na przykład Windows jako pierwszy target.
- Ustalić, czy płatności są w MVP czy dopiero po walidacji produktu.

ARTEFAKTY I KRYTERIA WYJŚCIA

- Jednostronicowy scope MVP i lista rzeczy poza MVP.
- Lista user stories krytycznych dla pierwszej wersji.
- Decyzja produktowa: co znaczy "działa" dla pierwszej sesji treningowej.
- Brak otwartych sporów o funkcje, które zatrzymują implementację.

ETAP 02

Uszczegółowienie architektury systemu

Przełożenie wysokopoziomowych założeń na konkretną architekturę aplikacji, repozytoriów i interfejsów.

PRACE DO WYKONANIA

- Rozpisać granice odpowiedzialności: web, backend, local engine, content pipeline.
- Zamknąć protokół WebSocket: handshake, wersjonowanie, heartbeat, eventy sesji.
- Określić model danych: użytkownik, trening, sesja, wynik, streak, postęp.
- Wybrać stos technologiczny dla każdej warstwy i ustalić standardy kodu.
- Zdecydować o strukturze CI, release flow i środowiskach.

ARTEFAKTY I KRYTERIA WYJŚCIA

- Diagram architektury systemu i przepływu danych.
- Specyfikacja komunikatów engine-web gotowa do implementacji.
- Model danych oraz lista endpointów backendu.
- Brak niejednoznaczności, kto liczy scoring i kto przechowuje wyniki.

ETAP 03

Przygotowanie repozytorium i infrastruktury developerskiej

Zbudowanie solidnej bazy pracy, żeby kolejne etapy były powtarzalne i kontrolowalne.

PRACE DO WYKONANIA

- Utworzyć repo albo monorepo z podziałem na web, backend, engine i shared contracts.
- Skonfigurować linting, formatowanie, pre-commit hooks i podstawowe testy.
- Postawić CI dla buildów, testów i artefaktów release.
- Dodac mechanizm wersjonowania kontraktów i release notes.
- Spisać README techniczne i sposób uruchamiania lokalnego.

ARTEFAKTY I KRYTERIA WYJSCIA

- Nowy developer jest w stanie uruchomić środowisko bez zgadywania.
- Push do głównej gałęzi uruchamia testy i buildy.
- Istnieje minimum jedna powtarzalna komenda release dla web i engine.

ETAP 04

Implementacja podstaw local engine

Zbudować działający proces lokalny, który potrafi czytać audio i komunikować się z webem.

PRACE DO WYKONANIA

- Stworzyć proces natywny z obsługą urządzeń audio, konfiguracji i lifecycle.
- Wdrożyć audio thread, processing thread i network thread.
- Dodac lock-free buffer, timestampowanie i podstawowe eventy sesyjne.
- Uruchomić lokalny serwer WebSocket z handshake i heartbeat.
- Przygotować logowanie diagnostyczne poza audio threadem.

ARTEFAKTY I KRYTERIA WYJSCIA

- Engine uruchamia się lokalnie i jest wykrywany przez aplikację webową.
- Silnik raportuje stan urządzenia, wersję i gotowość do sesji.
- Brak blokowania wątku audio przy zwykłym obciążeniu.

ETAP 05

Implementacja algorytmów audio i scoringu MVP

Zamknąć najważniejszą wartość produktu: rozpoznanie zagrania i uczciwy feedback w czasie zbliżonym do rzeczywistego.

PRACE DO WYKONANIA

- Dodac preprocessing: high-pass, noise gate, podstawowa normalizacja.
- Wdrożyć onset detection dla rozpoczęcia dźwięku.
- Wdrożyć pitch detection dla pojedynczej nuty.
- Zaprojektować prosty model scoringu: timing, poprawna nuta, pewność detekcji.

- Dodac kalibracje opoznienia i zapis parametrow per uzytkownik lub urzadzenie.
- Zbudowac zestaw probek testowych i benchmark dokladnosci.

ARTEFAKTY I KRYTERIA WYJSCIA

- Silnik zwraca eventy z nuta, timestampem i ocena trafienia.
- Scoring jest deterministyczny dla tych samych danych wejsciowych.
- Istnieja testy regresji dla sygnalow audio i znanych przypadkow granicznych.
- Dokladnosc i latencja sa wystarczajace do testow z realnymi gitarzystami.

ETAP 06

Warstwa backendu i danych

Zapewnic trwale dane produktu, autoryzacje i zapis wszystkiego, co potrzebne do progresu i analityki.

PRACE DO WYKONANIA

- Postawic baze danych i migracje.
- Wdrozyc auth, sesje i podstawowe profile uzytkownika.
- Dodac modele: training program, lesson, attempt, score summary, streak, achievements.
- Przygotowac API dla pobierania treningow, zapisu wynikow i dashboardu.
- Dodac audyt bledow, rate limiting i walidacje payloadow.

ARTEFAKTY I KRYTERIA WYJSCIA

- Pelna sesja treningowa zapisuje wynik i jest widoczna po odswiezeniu.
- Dane sa spójne miedzy webem a backendem.
- Istnieje plan backupu i odtworzenia srodowiska.

ETAP 07

Frontend aplikacji webowej

Zbudowac warstwe, ktora prowadzi uzytkownika od wejscia do produktu do zakonczenia pierwszej sesji.

PRACE DO WYKONANIA

- Zaprojektowac landing, logowanie, onboarding, dashboard i ekran treningu.
- Dodac wykrywanie local engine, status polaczenia i instrukcje dla uzytkownika.
- Przygotowac widok kalibracji oraz test urzadzenia audio.
- Pokazac scoring na zywo i ekran podsumowania sesji.
- Dodac stany bledow: brak engine, brak mikrofonu, niekompatybilna wersja, utrata polaczenia.

ARTEFAKTY I KRYTERIA WYJSCIA

- Nowy użytkownik przechodzi przez flow bez pomocy developera.
- Interfejs jasno komunikuje co trzeba zrobić, gdy lokalny silnik nie działa.
- UI nie blokuje sesji i nie przejmuje odpowiedzialności za scoring.

ETAP 08

Content, ekonomia produktu i mechaniki progresu

Zrobić z technologii produkt, do którego chce się wracać codziennie, a nie tylko demonstrację audio.

PRACE DO WYKONANIA

- Stworzyć pierwszy katalog treningów i ścieżek rozwoju.
- Określić zasady zdobywania punktów, streaków, poziomów i nagród.
- Przygotować copy onboardingowe i komunikaty motywacyjne.
- Jeśli płatności są w MVP, zdefiniować plany, limity i paywall.
- Zaprojektować panel administracyjny lub format danych do dodawania nowych treningów.

ARTEFAKTY I KRYTERIA WYJSCIA

- Produkt ma zawartość wystarczającą na pierwsze tygodnie testów.
- Mechanika progresu wspiera codzienny nawyk, a nie tylko pojedynczy wynik.
- Dodanie nowego treningu nie wymaga zmian w kodzie engine.

ETAP 09

Packaging, aktualizacje i dystrybucja local engine

Doprowadzić silnik lokalny do postaci produktu, który można bezpiecznie rozdawać i aktualizować.

PRACE DO WYKONANIA

- Stworzyć instalator, podpisywanie binarek i mechanizm auto-update lub update prompt.
- Dodać zgodne wersjonowanie z aplikacją webową oraz wymuszanie kompatybilności protokołu.
- Przygotować telemetrię awarii startu i problemów z audio device.
- Opracować strony pobierania, instrukcje instalacji i FAQ.
- Przetestować upgrade z jednej wersji engine do kolejnej bez utraty ustawień.

ARTEFAKTY I KRYTERIA WYJSCIA

- Istnieje stabilny pakiet instalacyjny dla pierwszej wspieranej platformy.
- Proces aktualizacji nie rozbija sesji i nie miesza wersji protokołu.

- Zespół zna najczęstsze problemy instalacyjne i ma gotowe odpowiedzi.

ETAP 10

Jakość, testy systemowe i hardening

Usunąć problemy, które w zamkniętym zespole wydają się małe, ale po launchu natychmiast zabijają adopcję.

PRACE DO WYKONANIA

- Uruchomić testy jednostkowe, integracyjne i end-to-end dla kluczowych flow.
- Przygotować test matrix: system operacyjny, karty audio, mikrofony, interfejsy, przeglądarki.
- Zweryfikować zachowanie przy słabych warunkach: brak dostępu do mikrofonu, duża latencja, zerwanie połączenia.
- Zbadać obciążenie backendu, limity websocketów i logikę sesji równoległych.
- Wprowadzić checklisty release, smoke tests i rollback plan.

ARTEFAKTY I KRYTERIA WYJŚCIA

- Krytyczne scenariusze mają automatyczne pokrycie albo jawna procedura manualna.
- Lista znanych błędów jest zamknięta i posegregowana na blocker, major, minor.
- Nie ma otwartych blockerów dla onboardingów, sesji treningowej i zapisu wyniku.

ETAP 11

Beta zamknięta i walidacja z użytkownikami

Sprawdzić, czy produkt działa poza zespołem i czy ludzie faktycznie rozumieją jego wartość.

PRACE DO WYKONANIA

- Zrekrutować pierwszą grupę beta testerów o różnym poziomie gry i sprzeczności.
- Przygotować scenariusz testowy: instalacja, pierwsza sesja, druga sesja po 24 godzinach, feedback.
- Zbierać dane ilościowe: aktywacja, procent ukończonych onboardingów, liczba sesji, czas do pierwszego sukcesu.
- Zbierać dane jakościowe: gdzie użytkownicy się gubią, kiedy przestają ufać scoringowi, co ich motywuje.
- Regularnie zamykać pętle poprawek w krótkich iteracjach.

ARTEFAKTY I KRYTERIA WYJŚCIA

- Istnieje raport z beta testów z listą decyzji produktowych i technicznych.

- Wiadomo, co psuje retencje i co trzeba poprawić przed publicznym startem.
- Minimum kilkunastu użytkowników dochodzi do zakończonej sesji bez wsparcia 1:1.

ETAP 12

Operacje, support i gotowosc do launchu

Przygotować zespół i systemy na moment, w którym liczba problemów rośnie razem z ruchem.

PRACE DO WYKONANIA

- Skonfigurować monitoring aplikacji, backendu, engine i procesu instalacji.
- Dodac alerty dla spadku aktywacji, wzrostu crashy i awarii websocketów.
- Stworzyć panel supportowy albo minimum procedures do analizy zgłoszeń.
- Przygotować politykę prywatności, regulamin, zgody i komunikaty prawne.
- Spisać runbook incydentowy i plan komunikacji z użytkownikami przy awarii.

ARTEFAKTY I KRYTERIA WYJSCIA

- Zespół wie kto reaguje na konkretny typ awarii.
- Metryki sukcesu launchu są zdefiniowane i widoczne w dashboardach.
- Istnieje gotowy plan rollbacku i czas reakcji na incydent.

ETAP 13

Soft launch

Wypuścić produkt do ograniczonej grupy publicznej, zanim ruch i oczekiwania stana się zbyt duże.

PRACE DO WYKONANIA

- Uruchomić ograniczoną rejestrację lub listę oczekujących.
- Włączyć kampanie testowe tylko na tyle, ile zespół umie obsłużyć.
- Codziennie przeglądać wskaźniki aktywacji, retencji i crashy.
- Priorytetyzować poprawki blokujące i wprowadzać małe, szybkie releasy.
- Weryfikować, czy support nadaje i czy instalacja nie jest główną barierą.

ARTEFAKTY I KRYTERIA WYJSCIA

- Produkt utrzymuje stabilność przy realnym ruchu.
- Kluczowe wskaźniki nie pogarszają się po zwiększeniu liczby użytkowników.
- Nie ma nowych blockerów, których nie widac było w beczie.

ETAP 14

Publiczny launch i pierwsze 30 dni po starcie

Wypuscic produkt szeroko, nie tracac kontroli nad jakoscia, supportem i rytmem poprawek.

PRACE DO WYKONANIA

- Odblokowac publiczna rejestracje, komunikacje marketingowa i publikacje launchowe.
- Monitorowac launch hour by hour, potem daily.
- Zamykac incydenty i publikowac poprawki w stalej kadencji.
- Uruchomic proces zbierania roadmapy po starcie na podstawie danych, a nie przeczac.
- Po 30 dniach zrobic review: techniczne, produktowe i finansowe.

ARTEFAKTY I KRYTERIA WYJSCIA

- Launch jest zakonczony bez awarii krytycznej lub ma opanowany plan naprawczy.
- Istnieje lista najwazniejszych zmian do wersji 1.1 i 1.2.
- Zespol ma jasnosc, czy produkt przechodzi z fazy walidacji do fazy wzrostu.

Przekrojowe obszary, ktore musza byc robione przez caly projekt

OBSZAR	CO ROBIC STALE	PO CZYM POZNAC, ZE JEST ZANIEDBANY
Bezpieczenstwo	Walidacja wejsc, zarzadzanie sekretami, aktualizacje zaleznosci, ograniczanie uprawnień.	Losowe awarie auth, podatnosc na naduzycia, brak pewnosc co trafia do logow.
Observability	Logi, tracing, metryki, crash reports, dashboardy per warstwa.	Problemy zgłaszane przez uzytkownikow sa nieodtworzalne.
Jakosc produktu	Regularne przeglady UX, smoke testy i porzadkowanie backlogu.	Zespol poprawia przypadkowe szczegoly zamiast przyczyn porzucania produktu.
Dokumentacja	Aktualizacja kontraktow, setupu, procedur release i supportu.	Nowe osoby potrzebuja ustnych wyjasnien do kazdego kroku.

Najwazniejsza zasada wykonawcza: jesli trzeba wybierac miedzy "ladniejszym UI" a "stabilnym timingiem, scoringiem i komunikacja engine-web", zawsze wygrywa rdzen. Bez zaufanego feedbacku audio cala reszta produktu traci sens.

Dokument przygotowany jako szczegółowy plan realizacji i wdrożenia dla projektu RiffRush.